

SELENIUM

- Widely used automation tool which primarily developed in JS – Thoughtworks Company
- Opensource and platform independent
- Can write in any lang: Java, python, csharp..

Selenium Suite:



Selenium IDE:

- Browser plugin
- Records and exports as reusable scripts

Selenium RC:

- Multi programming and multi browser support
- Depreciated in Selenium 2
- After 1.2 version deprecated in 2.0 version and newly introduced WebDriver

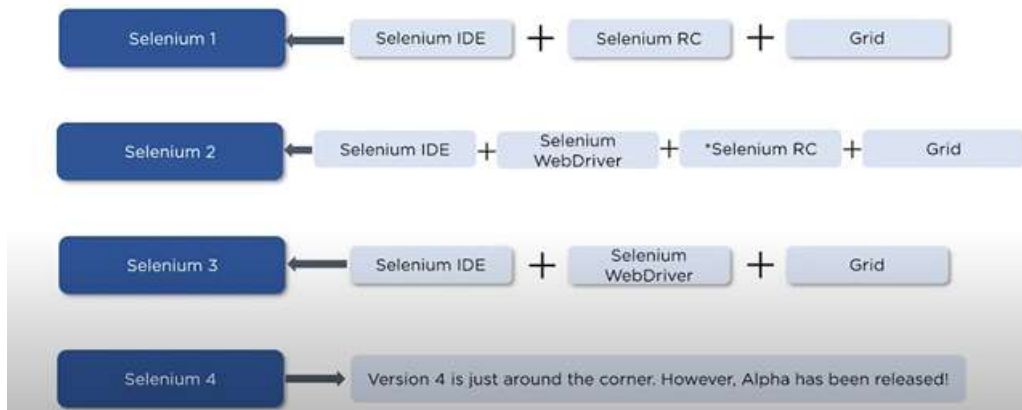
Selenium WebDriver:

- Uses API's to interact with browser
- Successor of Selenium RC
- Directly interacts to browser instead of RC

Selenium Grid:

- Runs Multiple tests in diff browsers
- Capable of running tests in parallel

Selenium suite



Locators:

- id
- name
- className
- Link text
- Partial link text
- text
- xpath
- css

Dropdowns:

```
Select dpdown = new Select();
List<WebElement> options = dpdown.getOptions();
for(WebElement element : options){
    syso(element);
}
dpdown.selectByIndex(1);
dpdown.selectByValue("java");
dpdown.selectByVisibleText("JavaScript");
dpdown.getFirstSelectedOption().getText();
List<WebElement> allOptions=dpdown.getAllSelectedOptions();
for(WebElement options : allOptions){
    syso(options);
}
```

Navigation methods:

```
WebDriverManager .chromedriver().setup();
WebDriver driver = new ChromeDriver();
driver.manage().window().maximize();
driver.get("url");
```

```
driver.navigate().To("url")
driver.navigate().back();
driver.navigate().refresh();
driver.navigate().forward();
```

Handle Alerts:

```
driver.switchTo().alert().getText();
driver.switchTo().alert().accept();// ok
driver.switchTo().alert().dismiss();// cancel
driver.switchTo().alert().sendKeys("Hello");// for prompt
```

- Alert Box: No yes/no options, only message displaying box
- Confirm Box: yes/No box
- Prompt Box: Text field contains in box

Alerts are not present in html code, so unable to handle like other elements, can't inspect directly

Window Handles:

//singleWindow and same for single tab

```
driver.get("url");//pass url
String parentWindow = driver.getWindowHandle();//gets current window handle
Set<String> windowHandles = driver.getWindowHandles();//stores all window
handles in set
for(String windowHandle = windowHandles){for each loop
if(!windowHandle.equals(parentWindow)){//checking for not a parent window
driver.switchTo.window(windowHandle);//switching to child window
driver.manage().window().maximize();//maximizing
driver.close();//closing current existing window
}}
driver.switchTo().window(parentWindow);//switching to parent window
driver.quit();//quitting driver instance from all windows
```

Frames:

<frame> - upto HTML4 //like child for frameset
<frameset> - to create multiple frames as frameset //like parent from frame
<iframe> - introduced from HTML5 //replacement for both frame and frameset using inlineframe

```
driver.switchTo.frame("frm1"); //switches to frame1
driver.switchTo.defaultContent(); //switches to main frame
driver.switchTo.frame("frm2"); //again to frame2
driver.switchTo.frame("frm1"); //then to frame1
driver.switchTo.defaultContent(); //finally to main
```

Screenshots:

//If we use

```
ChromeDriver driver = new ChromeDriver(); or
RemoteDriver driver = new ChromeDriver(); //below lines can work
```

```

File sourceFile = driver.getScreenshotAs(OutputType.File);
File destFile = new File("./Screenshots/img.png");
FileUtils.copy(sourcefile,destFile);

//else
WebDriver driver = new ChromeDriver(); // need to typecast

TakesScreenshot takeScreenshot = (TakesScreenshot) driver;//typecasting
File sourceFile = takeScreenshot.getScreenshotAs(OutputType.File);
File destFile = new File("./Screenshots/img.png");
FileUtils.copy(sourcefile,destFile);

```

PageLoadTimeout:

default timeout – 300sec

Exception – Timeout exception

```
driver.manage().timeouts().pageLoadTimeout(5, TimeUnit.SECONDS);
```

ImplicitWaitTimeout:

waitTimeout applicable for entire browser session

```
driver.manage().implicitlyWait(6, TimeOut.SECONDS);
```

// implicitly wait 6 seconds for all elements in script, if element loads in 1 or 2 secs then wont wait for remaining seconds

ExplicitWaitTimeout:

To overcome Synchronization issues... Dynamic wait .. not applicable for entire session

Default interval wait = 500 milliseconds

WebDriverWait:

```
WebDriverWait wait = new WebDriverWait(driver,10)
```

```
wait.until(d->d.findElement(By.id("un")).isDisplayed());
```

```
wait.until(ExpectedConditions.visibilityOfElementLocated(By.id("username")));
```

//by default cooling period will be 500ms and we can create our own function instead of expected conditions which is commonly used

FluentWait

```
FluentWait<Webdriver> wait = new FluentWait<WebDriver>(driver);
```

```
wait.withTimeout(Duration.ofSeconds(10));
```

get App source

URL – driver.getCurrentUrl();

Title – driver.getTitle();

PageSource – driver.getPageSource();

get Web element Properties:

getTagName, get HTML attribute value, get text, CSS value using Selenium WebDriver

- `driver.findElement(By.id("")).getTagName();` //input
- `driver.findElement(By.id("")).getAttribute("type");` //submit
- `driver.findElement(By.id("")).sendKeys("Password");` //passing value in text box
- `driver.findElement(By.id("")).getAttribute("value");` //Password – getting value
- `driver.findElement(By.id("")).getText();` //Hello
- `driver.findElement(By.id("")).getCssValue("background-color");` //get css value in rgba format
- `driver.findElement(By.id("")).getCssValue("display");` //block

Action:

Won't directly performs actions. need `build().perform()`

Mouse hover – `MoveToElement`

Click

DoubleClick

right Click or Context click

`Actions action = new Actions();`

`actions.moveToElement(driver.findElement(By...)).click().perform();`

`actions.doubleClick(driver.findElement(By...)).perform();`

`actions.contextClick(driver.findElement(By...)).perform();`

Calender Controls:

JavaScript Executor:

It's an interface in selenium webdriver

static:

id

name

classname

tagname

dynamic:

css selector

xpath

Selenium manager:

Selenium 4.6.0 and above... inbuilt feature like playwright and cypress

Automatically downloads and sets driver file instead of 3rd party Webdriver Manager

XPATH:

Absolute: starts from root - /

Relative: starts from any point – //tagname[@attribute='value']

Xpath using Attributes - //tagname[@attribute='value']

Xpath Operators – equals operator, <, >

Xpath Conditions – and, or, not -

//input[@class='class' and @id='first']

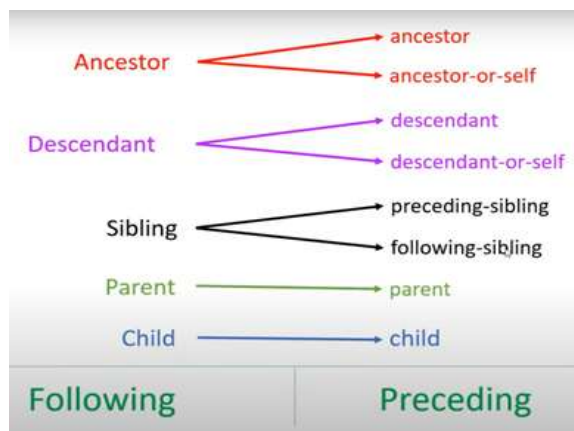
//input[not(@class='class')] and //input[@class!='class']

Xpath with Index - (//table[@id='contactList']/tbody/tr/td)[1] gives unique element

Xpath Functions –

- Text: //a[text()='Signin'] needs to pass full text
- Contains: //a[contains(text(),'account')] or //div[contains(@class,signin)]
- Normalized – space: //a[normalize-space(text())='FirstName'] ignores space & check with text
- Last: (//table[@id='contactList']/tbody/tr/td)[last()] gives last search
- Starts with: //a[starts-with(text(),'signin')]
- Position: - (//table[@id='contactList']/tbody/tr/td)[position()=2] same works like index

Xpath Axes:



Xpath Shortcuts:

- for text() we can replace it with "."
//a[text()='Signin'] = //a[., 'Signin']
- for child instead of writing word as child single slash is enough
//div[@class='signin']/class::h1 = //div[@class='signin']/h1
- For parent can be replaced with "/" points to immediate parent
- For descendant = "//"